IT305 Lab: Socket Programming Project — Master Implementation, File Location & Git PR Plan (Group 103 & 106)

Submission Target: FinalSubmission_Group_103_106.tar.gz

Extended Final Deadline: October 23, 2026

1. Day-by-Day Schedule, Execution Order & Git Pull Request (PR) Workflow (Oct 15 – Oct 23)

Because all 6 members work in a single shared Git repository using Pull Requests (PRs), you do **not** need anyone to sit idle waiting for another person. On **Day 1 (Oct 15)**, you merge common_types.h (PR #0) into main first. Immediately after PR #0 is merged, **all 6 members create their own feature branches and start coding in parallel on Day 1**.
To guarantee zero merge conflicts and a clean dependency chain, **you merge PRs into main in 5 sequential waves**:

Wave & Dates	Active Members & Parallel Coding Work	Exact PR Merge Order & When to Merge
Wave 0 (Day 1 Morning) **Oct 15, 2026**	**All 6 Members:** Initialize the Git repository with the 5 final folders (Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/, Results/) and commit common_types.h into the 4 code folders.	**Merge PR #0 (init-repo-header) first on Oct 15.** Once merged, all 6 members run git pull origin main and start coding on their own branches (member1-server, member2-protocol, etc.) on Day 1.
Wave 1 (Days 1–3) **Oct 15 – Oct 17, 2026**	• **Member 2** codes & unit-tests protocol.h/.cpp. • **Member 6** codes & unit-tests metrics_logger.h/.cpp and the 4 Makefile scripts. • **Member 1** codes &	**Merge in this exact order on Oct 17:** 1. **PR #1 (Member 2 — member2-protocol):** Merges protocol.h/.cpp first because both server and client depend on send_frame() and

	unit-tests directory_mapper.h/.cpp, concurrency.h/.cpp, and server.cpp. *(Meanwhile, Members 3, 4, and 5 code their modules on their own branches against common_types.h)*.	recv_frame(). 2. **PR #2 (Member 6 — member6-logger-build):** Merges metrics_logger.h/.cpp and the 4 Makefile scripts second so logging and builds are active. 3. **PR #3 (Member 1 — member1-server-core):** Member 1 pulls main (getting Member 2 & Member 6's files) and merges directory_mapper.h/.cpp, concurrency.h/.cpp, and server.cpp.
Wave 2 (Days 4–5) **Oct 18 – Oct 19, 2026**	• **Member 3** finishes fault.h/.cpp and client.cpp, pulls main (which now has Members 1, 2, and 6), and tests **Part 1** and **Part 2 Case 1** end-to-end. • **Member 4** & **Member 5** finish unit-testing their standalone modules.	**Merge on Oct 19:** 4. **PR #4 (Member 3 — member3-fault-client):** Merges fault.h/.cpp and client.cpp into main. *Milestone Verified:* Running make inside Part1/ and Part2_Case1/ now produces working server and client binaries.
Wave 3 (Days 6–7) **Oct 20 – Oct 21, 2026**	• **Member 4** pulls main (with Wave 1 & Wave 2 complete), hooks session.h/.cpp and checkpoint.h/.cpp into Part2_Case2/ and Part2_Case2_Enhanced/, and tests all 7 resume validation checks.	**Merge in this exact order on Oct 21:** 5. **PR #5 (Member 4 — member4-session-ckpt):** Merges session.h/.cpp and checkpoint.h/.cpp first (since Case 2 Enhanced also uses create_session()).

	• **Member 5** pulls main and hooks enhanced_epoll_server.h/.cpp and enhanced_parallel_client.h/.cpp into Part2_Case2_Enhanced/.	6. **PR #6 (Member 5 — member5-enhanced):** Member 5 pulls main (getting Member 4's session code) and merges enhanced_epoll_server.h/.cpp and enhanced_parallel_client.h/.cpp into Part2_Case2_Enhanced/. *Milestone Verified:* All 4 folders (Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/) now build with zero warnings.
Wave 4 (Day 8) **Oct 22, 2026**	• **Member 6** (with team support on the 3 lab computers) runs run_benchmarks.sh across Node 1 (Server) and Nodes 2 & 3 (Clients) for $N \in \{1, 2, 4, 8, 16, 32\}$ and $P_f \in \{0.0, 0.1, 0.3, 0.5\}$, then runs plot_results.py.	**Merge on Oct 22 Evening:** 7. **PR #7 (Member 6 — member6-benchmarks):** Merges Results/*.csv, Results/*.png, run_benchmarks.sh, and plot_results.py into Results/.
Wave 5 (Day 9) **Oct 23, 2026**	• **Members 1–5** hand over their written Final Report sections (Sections 1–5) to **Member 6**. • **Member 6** writes Sections 6–9, compiles FinalReport_Group_103_106.pdf and README.md, and packages the final archive.	**Merge on Oct 23 Morning:** 8. **PR #8 (Member 6 — member6-final-report):** Merges FinalReport_Group_103_106.pdf and README.md into the root folder, runs make clean before packaging or verifies builds, and creates FinalSubmission_Group_103_106.tar.gz.

2. Exact Repository Folder Layout & Where to Save Every File

Do **not** create any loose or temporary folders. Every member saves their assigned files directly into the exact target submission folders shown below:

Owner	Assigned File Names	Exact Target Folders Where File Must Be Saved	Git PR Wave
All (Shared)	common_types.h	Save in **all 4 code folders**: • Part1/common_types.h • Part2_Case1/common_types.h • Part2_Case2/common_types.h • Part2_Case2_Enhanced/common_types.h	PR #0 (Oct 15)

Member 1	directory_mapper.h directory_mapper.cpp concurrency.h concurrency.cpp server.cpp	Save in **all 4 code folders**: • Part1/ • Part2_Case1/ • Part2_Case2/ • Part2_Case2_Enhanced/	PR #3 (Wave 1 — Oct 17)
Member 2	protocol.h protocol.cpp	Save in **all 4 code folders**: • Part1/ • Part2_Case1/ • Part2_Case2/ • Part2_Case2_Enhanced/	PR #1 (Wave 1 — Oct 17)

Member 3	fault.h fault.cpp client.cpp	Save in **all 4 code folders**: • Part1/ • Part2_Case1/ • Part2_Case2/ • Part2_Case2_Enhanced/	PR #4 (Wave 2 — Oct 19)
Member 4	session.h session.cpp checkpoint.h checkpoint.cpp	Save in **Case 2 & Case 2 Enhanced folders**: • Part2_Case2/ • Part2_Case2_Enhanced/	PR #5 (Wave 3 — Oct 21)

Member 5	enhanced_epoll_server.h enhanced_epoll_server.cpp enhanced_parallel_client.h enhanced_parallel_client.cpp	Save **only in Case 2 Enhanced folder**: • Part2_Case2_Enhanced/	PR #6 (Wave 3 — Oct 21)
Member 6	metrics_logger.h metrics_logger.cpp Makefile	Save in **all 4 code folders**: • Part1/ (Makefile uses -DPART_NUM=1 -DCASE_MODE=0) • Part2_Case1/ (Makefile uses -DPART_NUM=2 -DCASE_MODE=1) • Part2_Case2/ (Makefile uses -DPART_NUM=2 -DCASE_MODE=2) • Part2_Case2_Enhanced/ (Makefile uses -DPART_NUM=2 -DCASE_MODE=3)	PR #2 (Wave 1 — Oct 17)

Member 6	run_benchmarks.sh plot_results.py Part1_metrics.csv Part2_metrics.csv Part1_SingleVsMulti_throughput.png Part2_AllCases_throughput.png Part2_AllCases_overhead.png	Save inside Results/ **folder**: • Results/	PR #7 (Wave 4 — Oct 22)
Member 6 *(with report text from Members 1–5)*	FinalReport_Group_103_106.pdf README.md	Save inside **Root Submission Folder**: • FinalSubmission_Group_103_106/FinalReport_Group_103_106.pdf • FinalSubmission_Group_103_106/README.md	PR #8 (Wave 5 — Oct 23)

3. Shared Header Contract (common_types.h — Commit in PR #0 on Day 1)

Save this exact file in Part1/common_types.h, Part2_Case1/common_types.h, Part2_Case2/common_types.h, and Part2_Case2_Enhanced/common_types.h on Day 1 so every member can compile and unit-test their .cpp files independently:

Available in https://github.com/nisargvekariya01/CN_Project/tree/main

4. Detailed Member-by-Member Work Breakdown

Member 1

- **Git Branch & Merge Wave:** Branch member1-server-core — Starts coding on **Day 1 (Oct 15)**, opens and merges **PR #3 in Wave 1 (Oct 17)** right after Member 2 (PR #1) and Member 6 (PR #2).
- **Assigned Files & Exact Save Locations:**
  - directory_mapper.h and directory_mapper.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
  - concurrency.h and concurrency.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
  - server.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
- **Assigned Final Report Section:** Section 1 (*Server Data Structures, Directory Mapping, CLI Parsing & Concurrency Execution Loops*).
- **How to Test Independently on Day 1–2:** Run a standalone test binary ./test_mapper <topics_root_dir> to verify all 10 Animals-10 directories load into topic_map with deterministically sorted files, and test run_sequential_loop() and run_concurrent_loop() using nc (netcat).

Functions to Implement (Member 1)

1. **int parse_server_cli(int argc, char* argv[], int part_num, uint16_t* out_port, std::string* out_root_dir, double* out_pf, int* out_threaded_mode, uint32_t* out_seed)**

   - **Input Parameters:** argc, argv from main(), part_num (1 for Part I, 2 for Part II), and output pointers for port, root_dir, failure probability pf, threaded_mode (0 = single-threaded, 1 = multi-threaded), and seed.
   - **Expected Output:** Returns 0 on valid CLI arguments; returns -1 and prints usage to stderr if invalid.
   - **Internal Logic:**
     - Strictly enforces the auto-grader positional CLI: ./server <port> <topics_root_dir> for Part I (argc >= 3) and ./server <port> <topics_root_dir> <failure_probability> for Part II (argc

>= 4).
- Validates port in [1024, 65535] via strtol(), verifies <topics_root_dir> exists and is a directory (S_ISDIR(st.st_mode)) via stat(), and for Part II validates pf in [0.0, 1.0] via strtod().
- Parses optional trailing flags (--single-threaded to select sequential mode in Part I, --seed <val>) while keeping multi-threaded mode as default.

2. **int scan_and_map_topics(const std::string& root_dir, std::unordered_map<std::string, TopicEntry>& topic_map)**

   - **Input Parameters:** root_dir (<topics_root_dir> from CLI) and reference to the in-memory std::unordered_map<std::string, TopicEntry>.
   - **Expected Output:** Returns the count of valid topics indexed (>= 0), or -1 if opendir(root_dir) fails.
   - **Internal Logic:**
   - Calls opendir(root_dir.c_str()) and iterates over entries via readdir(), skipping "." and "..".
   - For each valid subdirectory verified via stat(), uses the folder name (e.g., "Cane", "Cavallo") as the hash map key.
   - Scans all regular files (S_ISREG) inside the topic folder, queries each file's byte size via stat(), stores them in TopicEntry.files, and sorts TopicEntry.files lexicographically by filename (std::sort) so file_index (0 .. file_count - 1) is 100% deterministic across reconnects.
   - Sums total_bytes and inserts the TopicEntry into topic_map.

3. **const TopicEntry* get_topic_path(const std::unordered_map<std::string, TopicEntry>& topic_map, const std::string& topic)**

   - **Input Parameters:** Constant reference to topic_map and requested topic string.
   - **Expected Output:** Pointer to const TopicEntry if found in $O(1)$ average time; nullptr if not found.
   - **Internal Logic:** Performs a lock-free .find(topic) lookup on the read-only startup hash map.

4. **int create_listening_socket(uint16_t port) & void register_signal_handlers()**

   - **Input Parameters:** port (uint16_t).
   - **Expected Output:** Returns listen_fd >= 0 on success, -1 on failure.
   - **Internal Logic:** Creates a TCP socket (AF_INET, SOCK_STREAM), sets SO_REUSEADDR, binds to INADDR_ANY:port, calls listen(listen_fd, 128), ignores SIGPIPE, and registers sigaction for SIGINT and SIGTERM to set keep_running = 0 and close listen_fd for clean Valgrind shutdown.

5. **void run_sequential_loop(int listen_fd, const std::unordered_map<std::string, TopicEntry>& topic_map)**

   - **Input Parameters:** Listening socket listen_fd and topic_map.
   - **Expected Output:** void (exits cleanly when keep_running == 0).

- **Internal Logic:** Loops while keep_running, calling blocking accept(), executing process_client() directly on the main thread until the entire topic finishes transferring, and closing client_fd.

6. **void run_concurrent_loop(int listen_fd, const std::unordered_map<std::string, TopicEntry>& topic_map, double pf, int case_mode, uint32_t seed) & void* worker_thread(void* arg)**

   - **Input Parameters:** listen_fd, topic_map, failure probability pf (0.0 in Part I), case_mode (0, 1, or 2), and seed.
   - **Expected Output:** void.
   - **Internal Logic:** Loops on accept(). For each client, dynamically allocates WorkerArgs, sets PTHREAD_CREATE_DETACHED on pthread_attr_t, and spawns pthread_create(&tid, &attr, worker_thread, args). Inside worker_thread(), calls process_client() to dispatch OP_GET (all cases) or OP_RESUME (Case 2), closes client_fd, and frees WorkerArgs.

Member 2

- **Git Branch & Merge Wave:** Branch member2-protocol — Starts coding on **Day 1 (Oct 15)**, opens and merges **PR #1 first in Wave 1 (Oct 17)**.
- **Assigned Files & Exact Save Locations:**
  - protocol.h and protocol.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
- **Assigned Final Report Section:** Section 2 (*Client-Server Communication Protocol, 8-Byte Framing & Binary Payload Layouts*).
- **How to Test Independently on Day 1–2:** Create a local socketpair(AF_UNIX, SOCK_STREAM, 0, sv) unit test that serializes all 14 opcodes (0x01–0x0E) into sv[0], reads them from sv[1] via recv_frame(), and asserts 100% bit-exactness.

Functions to Implement (Member 2)

1. **ssize_t send_all(int sockfd, const void* buf, size_t len, uint64_t* wire_counter = nullptr) & ssize_t recv_all(int sockfd, void* buf, size_t len, uint64_t* wire_counter = nullptr)**

   - **Input Parameters:** Socket descriptor sockfd, buffer buf, exact byte count len, and optional wire_counter pointer to accumulate total wire bytes (B_{total}).
   - **Expected Output:** Returns len on complete transfer, 0 on premature peer EOF (in recv_all), or -1 on socket error (ECONNRESET, EPIPE).
   - **Internal Logic:** Loops around send(sockfd, ptr + total, len - total, MSG_NOSIGNAL) or recv(sockfd, ptr + total, len - total, 0) until total == len, retrying transparently if errno == EINTR and updating *wire_counter.

2. **int send_frame(int sockfd, uint8_t opcode, const uint8_t* payload, uint32_t payload_len, uint64_t* wire_counter = nullptr)**

   - **Input Parameters:** sockfd, opcode (0x01–0x0E), serialized payload pointer (nullptr if

payload_len == 0), payload_len (<= 1 MB), and optional wire_counter.

- **Expected Output:** Returns 0 on success, -1 on failure.
- **Internal Logic:** Packs an 8-byte stack buffer: bytes 0..1 = htons(0x4954), byte 2 = 0x01, byte 3 = opcode, bytes 4..7 = htonl(payload_len). Calls send_all(sockfd, hdr_buf, 8, wire_counter) followed by send_all(sockfd, payload, payload_len, wire_counter) if payload_len > 0.

3. **int recv_frame(int sockfd, FrameHeader* out_hdr, std::vector<uint8_t>& out_payload, uint64_t* wire_counter = nullptr)**

- **Input Parameters:** sockfd, output FrameHeader* out_hdr, reference to out_payload vector, and optional wire_counter.
- **Expected Output:** Returns 1 on valid frame received, 0 on clean/premature EOF, -1 on socket error, or -2 on PROTOCOL_ERROR.
- **Internal Logic:** Calls recv_all(sockfd, raw_hdr, 8, wire_counter). Unpacks magic = ntohs(...), version = raw_hdr[2], type = raw_hdr[3], payload_length = ntohl(...). Validates magic == 0x4954, version == 0x01, and payload_length <= MAX_FRAME_PAYLOAD (1 MB); if invalid, returns -2. Resizes out_payload to payload_length and calls recv_all() to read the exact payload.

4. **Big-Endian Payload Serializers & Parsers (Table 2 Layouts for Opcodes 0x01–0x0E)**

- Uses htons()/ntohs(), htonl()/ntohl(), and htobe64()/be64toh() with unpadded fixed-width integers and 2-byte length-prefixed strings:
- serialize_get / parse_get (0x01): [topic_len: uint16_t][topic: bytes].
- serialize_get_ok / parse_get_ok (0x02): [session_id: 16B][file_count: uint32_t][total_bytes: uint64_t] (session_id is 16 zero bytes in Part I and Case 1).
- serialize_status_msg / parse_status_msg (0x03 OP_ERROR & 0x0B OP_RESTART): [code: uint16_t][msg_len: uint16_t][reason: bytes].
- serialize_file_begin / parse_file_begin (0x04): [file_index: uint32_t][byte_offset: uint64_t][bytes_following: uint64_t][name_len: uint16_t][filename: bytes].
- serialize_file_data / parse_file_data (0x05): [file_index: uint32_t][offset: uint64_t][data_len: uint32_t][data: bytes].
- serialize_file_end / parse_file_end (0x06): [file_index: uint32_t].
- serialize_transfer_end / parse_transfer_end (0x07): [files_sent: uint32_t][useful_bytes: uint64_t].
- serialize_ckpt_ack / parse_ckpt_ack (0x08): [session_id: 16B][file_index: uint32_t][ack_offset: uint64_t].
- serialize_resume / parse_resume (0x09): [session_id: 16B][file_index: uint32_t][byte_offset: uint64_t][topic_len: uint16_t][topic: bytes][name_len: uint16_t][filename: bytes].
- serialize_resume_ok / parse_resume_ok (0x0A): [file_index: uint32_t][accepted_offset: uint64_t][rem_files: uint32_t][rem_bytes: uint64_t].
- serialize_get_enhanced / parse_get_enhanced (0x0C), serialize_join_session / parse_join_session (0x0D), and serialize_assign_work / parse_assign_work (0x0E).

Member 3

- **Git Branch & Merge Wave:** Branch member3-fault-client — Starts coding on **Day 1 (Oct 15)**, pulls main after Wave 1 merges, and opens/merges **PR #4 in Wave 2 (Oct 19)**.
- **Assigned Files & Exact Save Locations:**
  - fault.h and fault.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
  - client.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
- **Assigned Final Report Section:** Section 3 (*Chunk-Boundary Fault Injection Model & Part II Case 1 Stateless Recovery*).
- **How to Test Independently on Day 1–3:** Run a 10,000-chunk loop calling should_inject_fault(0.1, ...) to verify ~10% fault injection rate, and test run_client_part1_and_case1_loop() against a mock socket that drops mid-transfer.

Functions to Implement (Member 3)

1. **void init_thread_rng(uint32_t optional_seed)**

   - **Input Parameters:** optional_seed (0 for random seeding via std::random_device, or non-zero if --seed was passed on CLI).
   - **Expected Output:** void.
   - **Internal Logic:** Initializes a thread_local std::mt19937 rng_engine seeded by combining std::random_device{}(), pthread_self(), and CLOCK_MONOTONIC nanoseconds so concurrent worker threads never contend on a mutex or generate identical fault sequences.
2. **bool should_inject_fault(double pf, const char* topic, uint32_t file_idx, uint64_t bytes_sent)**

   - **Input Parameters:** Failure probability pf in [0.0, 1.0], topic name, current file_idx, and bytes_sent so far.
   - **Expected Output:** Returns true if the server worker thread must drop the connection immediately; false otherwise.
   - **Internal Logic:** Evaluated strictly at the boundary immediately after send_frame() finishes sending an OP_FILE_DATA chunk. If pf <= 0.0, returns false. Draws $r \sim Uniform(0.0, 1.0)$ on the thread_local RNG; if $r < P_f$, calls log_event("FAULT", ...) and returns true so the worker closes client_fd and exits.
3. **int stream_topic_to_client(int client_fd, const TopicEntry& entry, uint32_t start_file_idx, uint64_t start_offset, double pf, ServerSession* session_ptr, pthread_mutex_t* session_mutex)**

   - **Input Parameters:** client_fd, TopicEntry, starting file index and byte offset (0, 0 for Part I/Case 1; accepted_offset for Case 2), pf, and optional session_ptr + session_mutex.
   - **Expected Output:** Returns 0 on full OP_TRANSFER_END completion, -1 if fault-injected or client disconnected.
   - **Internal Logic:**

- Iterates i from start_file_idx to entry.files.size() - 1 and opens entry.files[i].full_path with fopen(..., "rb").
- If i == start_file_idx && start_offset > 0, seeks via fseeko(fp, start_offset, SEEK_SET).
- Sends OP_FILE_BEGIN (0x04) with bytes_following = file_size - offset.
- Reads up to 64 KB (CHUNK_SIZE) per iteration, sends OP_FILE_DATA (0x05), updates session_ptr->bytes_sent_highwater (under session_mutex if session_ptr != nullptr), and calls should_inject_fault(pf, ...). If true, closes fp and returns -1.
- Sends OP_FILE_END (0x06) after each file, marks session_ptr->all_files_sent = true after the final file, and sends OP_TRANSFER_END (0x07).

4. **int parse_client_cli(int argc, char* argv[], std::string* out_ip, uint16_t* out_port, std::string* out_topic, std::string* out_dir, int* out_max_retries, uint32_t* out_ckpt_win, int* out_streams)**

   - **Input Parameters:** argc, argv, and output pointers for <server_ip>, <server_port>, <topic>, <output_dir>, plus optional flags (--max-retries, --ckpt-window, --streams).
   - **Expected Output:** Returns 0 on valid CLI (argc >= 5), -1 otherwise.
   - **Internal Logic:** Enforces positional CLI ./client <server_ip> <server_port> <topic> <output_dir> and creates <output_dir>/<topic>/ via mkdir(..., 0755).

5. **int run_client_part1_and_case1_loop(const char* server_ip, uint16_t port, const char* topic, const char* output_dir, int max_retries, bool is_part1)**

   - **Input Parameters:** server_ip, port, topic, output_dir, max_retries (default 50), and is_part1.
   - **Expected Output:** Returns 0 on verified completion, or non-zero exit code if max_retries is exhausted.
   - **Internal Logic:**
     - Connects to the server and sends OP_GET <topic> (0x01).
     - Opens incoming files with O_WRONLY | O_CREAT | O_TRUNC (overwriting from Byte 0 on every retry).
     - If recv_frame() returns 0 (premature EOF before OP_TRANSFER_END) or -1 (ECONNRESET/EPIPE), adds all file bytes received in that failed attempt to redundant_bytes, closes the socket, sleeps for truncated exponential backoff (delay_ms = std::min(1000, 50 * (1 << attempt))), reconnects, and sends a fresh OP_GET <topic>.
     - Upon receiving OP_TRANSFER_END (0x07), verifies files_sent == files_received and useful_bytes == total_file_bytes_written, calls record_transfer_metrics(), and returns 0.

Member 4

- **Git Branch & Merge Wave:** Branch member4-session-ckpt — Starts coding on **Day 1 (Oct 15)**, pulls main after Wave 2 merges, and opens/merges **PR #5 first in Wave 3 (Oct 21)**.
- **Assigned Files & Exact Save Locations:**
  - session.h and session.cpp → Save in Part2_Case2/ and Part2_Case2_Enhanced/.
  - checkpoint.h and checkpoint.cpp → Save in Part2_Case2/ and Part2_Case2_Enhanced/.

- **Assigned Final Report Section:** Section 4 (*Part II Case 2 Session Management, Durability Invariant & 7-Step Validation*).
- **How to Test Independently on Day 1–4:** Write a unit test that creates a session, writes a .ckpt file via save_checkpoint(), simulates all 7 valid/invalid RESUME conditions (expired TTL, wrong topic, file_index == file_count "EOF" boundary, invalid offset, highwater exceeded), and verifies the exact RESUME_OK or RESTART response code.

Functions to Implement (Member 4)

1. **int create_session(const char* topic, uint8_t out_session_id[16], ServerSession** out_session_ptr)**

   - **Input Parameters:** Requested topic string, output 16-byte buffer out_session_id, and output pointer out_session_ptr.
   - **Expected Output:** Returns 0 on success, -1 on error.
   - **Internal Logic:** Reads 16 cryptographically random bytes from /dev/urandom into out_session_id. Locks session_table_mutex, inserts a ServerSession entry (file_index = 0, bytes_sent_highwater = 0, last_active = time(NULL), all_files_sent = false) keyed by the 32-char lowercase hex representation of session_id, and unlocks the mutex.
2. **int save_checkpoint(const char* output_dir, const CheckpointRecord* ckpt, int active_file_fd)**

   - **Input Parameters:** output_dir, pointer to CheckpointRecord, and active_file_fd (>= 0 if a target file is currently open).
   - **Expected Output:** Returns 0 on durable commit, -1 on I/O error.
   - **Internal Logic (Strict 3-Step Durability Sequence):**
     - **Step (a):** If active_file_fd >= 0, calls fsync(active_file_fd) so all payload bytes up to ckpt->byte_offset are physically on disk before updating the checkpoint.
     - **Step (b):** Formats the single-line ASCII record <session_id_hex32> <topic> <file_index> <byte_offset> <files_done> <file_name>\n into <output_dir>/.it305_<topic>.ckpt.tmp, calls fsync(tmp_fd), closes tmp_fd, and atomically calls rename(tmp_path, final_ckpt_path) (<output_dir>/.it305_<topic>.ckpt).
3. **bool load_checkpoint(const char* output_dir, const char* topic, CheckpointRecord* out_ckpt) & void remove_checkpoint(const char* output_dir, const char* topic)**

   - **Input Parameters:** output_dir, topic, and output pointer out_ckpt.
   - **Expected Output:** Returns true if a valid .it305_<topic>.ckpt exists and is parsed; false otherwise.
   - **Internal Logic:** Opens <output_dir>/.it305_<topic>.ckpt, parses the 6 space-separated fields, converts the 32-char hex string back into out_ckpt->session_id[16] (binary), and populates out_ckpt. remove_checkpoint() unlinks .it305_<topic>.ckpt on OP_RESTART or verified OP_TRANSFER_END.
4. **int validate_checkpoint(const CheckpointRecord* client_ckpt, const std::unordered_map<std::string, TopicEntry>& topic_map, uint16_t* out_err_code, std::string* out_reason, uint32_t* out_rem_files, uint64_t* out_rem_bytes,**

ServerSession out_session_ptr)**

- **Input Parameters:** client_ckpt parsed from OP_RESUME (0x09), topic_map, and output pointers for RESTART error code/reason or RESUME_OK remaining counts and ServerSession*.
- **Expected Output:** Returns 0 (OP_RESUME_OK 0x0A) if valid, or -1 (OP_RESTART 0x0B) if any check fails.
- **Internal Logic (Locks session_table_mutex, runs 600s lazy TTL sweep, and runs all 7 checks from Section 4.3):**

1. session_id exists in session_table (else 410 SESSION_EXPIRED).
2. now - session.last_active <= 600 seconds (else erase session & return 410 SESSION_EXPIRED).
3. client_ckpt->topic == session.topic (else 409 TOPIC_MISMATCH).
4. **Index Range Check (0 <= file_index <= file_count):**
   - **Terminal Case (file_index == file_count):** Valid iff byte_offset == 0, file_name == "EOF", and session.all_files_sent == true. Sets *out_rem_files = 0, *out_rem_bytes = 0, returns 0 (OP_RESUME_OK immediately followed by OP_TRANSFER_END).
   - If file_index > file_count, returns -1 (416 INVALID_OFFSET).
5. topic_entry.files[file_index].filename == client_ckpt->file_name (else 409 FILENAME_MISMATCH).
6. 0 <= byte_offset <= topic_entry.files[file_index].file_size (else 416 INVALID_OFFSET).
7. Cumulative bytes claimed by client <= session.bytes_sent_highwater (else 416 HIGHWATER_EXCEEDED).

5. **int open_and_seek_resume_file(const char* filepath, uint64_t accepted_offset) & int run_client_case2_loop(...)**

- **Input Parameters:** Target filepath under <output_dir>/<topic>/<file_name>, accepted_offset from OP_RESUME_OK, plus client CLI parameters.
- **Expected Output:** open_and_seek_resume_file returns fd >= 0 positioned at accepted_offset; run_client_case2_loop returns 0 on verified completion.
- **Internal Logic:** Calls open(filepath, O_WRONLY | O_CREAT, 0644) (never O_TRUNC or O_APPEND) and lseek(fd, accepted_offset, SEEK_SET) so any uncheckpointed tail (< 64 KB) is cleanly overwritten in place. Triggers save_checkpoint() every $W = 64KB$ and at every OP_FILE_END boundary (normalizing file_index = k + 1 or file_index = file_count, file_name = "EOF").

Member 5

- **Git Branch & Merge Wave:** Branch member5-enhanced — Starts coding on **Day 1 (Oct 15)**, pulls main after Member 4's PR #5 merges, and opens/merges **PR #6 in Wave 3 (Oct 21)**.
- **Assigned Files & Exact Save Locations:**
  - enhanced_epoll_server.h and enhanced_epoll_server.cpp → Save in Part2_Case2_Enhanced/.

- enhanced_parallel_client.h and enhanced_parallel_client.cpp → Save in Part2_Case2_Enhanced/.
- **Assigned Final Report Section:** Section 5 (*Planned & Implemented Performance Enhancements — Part II Case 2 Enhanced*).
- **How to Test Independently on Day 1–5:** Run the Enhanced server and client against a topic folder with $P_f = 0.3$ and $N = 4$ streams, verifying via md5sum or diff -r that files assembled via pwrite() match the source directory bit-for-bit.

Functions to Implement (Member 5)

1. **int set_socket_nonblocking(int sockfd) & int init_epoll_instance(int listen_fd)**

   - **Input Parameters:** Socket descriptor sockfd / listen_fd.
   - **Expected Output:** 0 on success for set_socket_nonblocking; epoll_fd >= 0 for init_epoll_instance.
   - **Internal Logic:** Uses fcntl(sockfd, F_SETFL, flags | O_NONBLOCK) and registers descriptors with epoll_create1(0) and epoll_ctl(epoll_fd, EPOLL_CTL_ADD, ...) using EPOLLIN | EPOLLET ($O(\log N)$ control operations and $O(K)$ ready-event notifications).
2. **ssize_t send_chunk_zerocopy(int client_fd, int file_fd, const ChunkWorkUnit& unit, uint64_t* wire_counter = nullptr)**

   - **Input Parameters:** client_fd, open file_fd, ChunkWorkUnit (file_index, file_offset, chunk_len <= 64 KB), and optional wire_counter.
   - **Expected Output:** Total payload bytes sent (chunk_len), or -1 on error/fault.
   - **Internal Logic:** Sends the 8-byte OP_FILE_DATA frame header + 16-byte chunk metadata prefix (file_index, offset, chunk_len) via send_all(), then transfers the file payload directly from kernel page cache to NIC buffer via sendfile(client_fd, file_fd, &off, chunk_len) in a loop handling EAGAIN/EWOULDBLOCK.
3. **bool pop_next_chunk_unit(EnhancedSessionState* sess, uint32_t conn_id, ChunkWorkUnit* out_unit)**

   - **Input Parameters:** Pointer to thread-safe EnhancedSessionState, sub-stream conn_id (0 .. N-1), and output out_unit.
   - **Expected Output:** true if an UNASSIGNED 64 KiB unit was assigned; false if all units are COMMITTED.
   - **Internal Logic:** Locks sess->queue_mutex, finds the next 64 KiB chunk in state CHUNK_UNASSIGNED (or requeues a CHUNK_IN_FLIGHT unit whose sub-stream dropped), transitions its state to CHUNK_IN_FLIGHT, and dispatches OP_ASSIGN_WORK (0x0E).
4. **int run_enhanced_client(const char* server_ip, uint16_t port, const char* topic, const char* output_dir, int num_streams, int max_retries) & void* client_parallel_worker(void* arg)**

   - **Input Parameters:** server_ip, port, topic, output_dir, num_streams (N = 4), and

max_retries.
- **Expected Output:** Returns 0 when all 64 KiB chunks across the topic are marked COMMITTED (1 in bitmask).
- **Internal Logic:**
  - Primary connection sends OP_GET_ENHANCED (0x0C) and receives session_id[16] and file metadata.
  - Pre-allocates target files via posix_fallocate(fd, 0, file_size) and spawns $N = 4$ worker threads (conn_id = 0..3).
  - Each worker connects a parallel TCP socket, sends OP_JOIN_SESSION (0x0D) carrying session_id, conn_id, and chunk_bitmap, and writes incoming 64 KiB chunks concurrently without lock contention using positional pwrite(fd, data, data_len, offset).
  - Updates the bitmask bit (chunk_bitmap[unit_id / 8] |= (1 << (unit_id % 8))). If one sub-stream drops due to a server fault, only that single thread reconnects and sends OP_JOIN_SESSION with the updated bitmap while the other 3 streams continue running uninterrupted (losing only 25% capacity temporarily under an equal-bandwidth assumption).

Member 6

- **Git Branch & Merge Wave:**
  - Branch member6-logger-build — Starts coding on **Day 1 (Oct 15)**, merges **PR #2 in Wave 1 (Oct 17)** (metrics_logger.h/.cpp and the 4 Makefile scripts).
  - Branch member6-benchmarks — Merges **PR #7 in Wave 4 (Oct 22)** (run_benchmarks.sh, plot_results.py, Results/*.csv, Results/*.png).
  - Branch member6-final-report — Merges **PR #8 in Wave 5 (Oct 23)** (FinalReport_Group_103_106.pdf and README.md).
- **Assigned Files & Exact Save Locations:**
  - metrics_logger.h and metrics_logger.cpp → Save in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/.
  - Makefile → Save in Part1/Makefile, Part2_Case1/Makefile, Part2_Case2/Makefile, Part2_Case2_Enhanced/Makefile.
  - run_benchmarks.sh, plot_results.py, Part1_metrics.csv, Part2_metrics.csv, and .png plots → Save in Results/.
  - README.md and FinalReport_Group_103_106.pdf → Save in Root Folder FinalSubmission_Group_103_106/.
- **Assigned Final Report Section:** Cover Page, Section 6 (*Extra Features*), Section 7 (*Code Architecture*), Section 8 (*Design Trade-offs*), and Section 9 (*Performance Analysis, Graphs & Observations*).
- **How to Test Independently on Day 1–2:** Call record_transfer_metrics() with sample values to verify Results/*.csv creation, and run plot_results.py to generate sample PNG graphs.

Functions & Scripts to Implement (Member 6)

1. **void record_transfer_metrics(const char* csv_filepath, const char* case_name, int num_clients, double pf, uint64_t useful_bytes, uint64_t total_bytes, uint64_t**

redundant_bytes, struct timespec t_start, struct timespec t_end, int retries)

- **Input Parameters:** Target CSV path (e.g., "../Results/Part2_metrics.csv"), case_name, concurrent client count num_clients (N), fault probability pf (P_f), useful_bytes (B_{useful}), total_bytes (B_{total}), redundant_bytes ($B_{redundant}$), CLOCK_MONOTONIC start/end timestamps, and retries.
- **Expected Output:** void (appends one thread-safe CSV row to Results/*.csv).
- **Internal Logic:**
  - Protects file access with a pthread_mutex_t.
  - Computes elapsed duration in seconds: $T_{complete} = (t_{end}.tv_sec - t_{start}.tv_sec) + (t_{end}.tv_nsec - t_{start}.tv_nsec) \times 10^{-9}$.
  - Computes the three core metrics defined in Section 9.1: $$\text{Goodput (MB/s)} = \frac{B_{\text{useful}}}{T_{\text{complete}} \times 10^6}, \quad \text{Wire Throughput (MB/s)} = \frac{B_{\text{total}}}{T_{\text{complete}} \times 10^6}, \quad \text{Overhead (\%)} = \left(\frac{B_{\text{total}} - B_{\text{useful}}}{B_{\text{useful}}}\right) \times 100$$
  - Appends case_name,num_clients,pf,t_start_sec,t_end_sec,duration_sec,useful_bytes,total_bytes, redundant_bytes,goodput_mbps,throughput_mbps,overhead_pct,retries\n to the CSV file.

2. **void log_event(const char* tag, const char* fmt, ...)**

- **Input Parameters:** tag ("INFO", "FAULT", "RESUME", "ERROR"), printf-style format string, and variadic arguments.
- **Expected Output:** void.
- **Internal Logic:** Formats a timestamped log line protected by log_mutex and writes to it305_runtime.log without blocking on interactive stdin prompts so the auto-grader never hangs.

3. **Makefiles & Benchmarking Automation (Makefile, run_benchmarks.sh, plot_results.py)**

- **Makefiles in Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/:** Each Makefile compiles with g++ -O2 -Wall -Wextra -Werror -pthread -std=c++17 (plus -DPART_NUM=1/2 -DCASE_MODE=0/1/2/3) and produces **only two executables in that exact directory**: server and client.
- **plot_results.py:** Reads Results/*.csv and generates:
  - Results/Part1_SingleVsMulti_throughput.png: Aggregate File Transfer Rate $G_{aggregate}(N) = \frac{\sum_{i=1}^{N} B_{useful,i}}{T_{experiment}}$ vs. $N \in \{1, 2, 4, 8, 16, 32\}$ for Single-Threaded and Multi-Threaded servers.
  - Results/Part2_AllCases_throughput.png & Results/Part2_AllCases_overhead.png:

Goodput, Wire Throughput, Completion Time, and Retransmission Overhead (%) across $P_f \in \{0.0, 0.1, 0.3, 0.5\}$ comparing Case 1, Case 2, and Case 2 Enhanced against Part I baselines.

5. Pre-Submission Auto-Grader Checklist (For Oct 23)

Before creating FinalSubmission_Group_103_106.tar.gz, verify these exact submission rules from IT305-lab-socket-programming-project-assignment.pdf:

1. **Exact Archive & Folder Layout:** FinalSubmission_Group_103_106/ containing FinalReport_Group_103_106.pdf, README.md, Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/, and Results/ (no loose files outside the root directory).
2. **Zero Compilation Warnings/Errors:** Run make clean && make inside all four code folders (Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/) and confirm each creates server and client in that same directory with zero warnings.
3. **Exact Positional CLI Verification:**
   - **Part I Server:** ./server <port> <topics_root_dir>
   - **Part II Server (All Cases):** ./server <port> <topics_root_dir> <failure_probability>
   - **Client (All Cases):** ./client <server_ip> <server_port> <topic> <output_dir> (writes received files to <output_dir>/<topic>/)